Assignment Guideline

Write a research report for C++ algorithms based on the requirements below.

1. Assignment Title Registration.

a) The leader must register the group member in the link HERE assignment registration form.
b) Due date: **18 May 2026 Monday (Week 8) before 5.00 pm**.
c) Maximum no of students: 5 persons.

2. Assignment Submission.

a) Softcopy (Submit to eBwise):
   i. Assignment Evaluation Form. (The leader of the group must submit the form).
   ii. Report file. Example GROUP 1.pdf (The leader of the group must submit the report).
   iii. Turnitin receipt. (The leader of the group must submit the receipt).
   iv. Self-Peer Evaluation form. (Each member must submit the form).
b) Due date: **1 January 2026 Monday (Week 10) before 5.00 pm**.

3. Assignment Turnitin Submission:

a) The lecturer will add a leader to the Turnitin web application based on the assignment registration form in week 3 or 4.
b) The leader will receive an email to register a Turnitin account. Refer to the **Guideline Turnitin.pdf** file.
c) The leader must submit the complete report in PDF file format to the Turnitin website.
d) The leader must submit the **Final Report**, **C++ code** and **Turnitin receipt** to eBwise.
e) The similarity report must be less than 20%.
f) The AI writing detection percentage must be less than 20%. You may use any free application to check your AI detection for your paper.
g) The mark will be deducted if more than 20% for similarity or AI writing detection.

4. Assignment Descriptions:

This assignment to enhance students' understanding of advanced searching and sorting algorithms through practical implementation, analysis, visualisation, and real-world problem-solving using C++, applying **Data Structure and Algorithms.** Students need to select TWO (2) types of Sorting **OR** TWO (2) types of Searching algorithms. In your report, the **2 algorithms** must **NOT** be in the lecture notes and **CANNOT** be the same with other groups.

Students are required to:

- Investigate non-lecture searching or sorting algorithms. Choose to do a search or a sorting algorithm. Example: Comparison between Radix Sort and Bucket Sort.
- Implement the algorithms manually without using built-in library algorithms, and analyse algorithm efficiency using datasets, compare algorithm performance and develop an interactive mini application that demonstrates the use of the algorithms in a meaningful scenario.
- The assignment emphasises problem-solving skills, algorithmic thinking, data analysis, program modularity and real-world application development.

5. Algorithm Restrictions:

Students are NOT allowed to use algorithms covered during lectures:

Searching Algorithms NOT Allowed

- Linear Search
- Binary Search
- Hashing Techniques

Sorting Algorithms NOT Allowed

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort

Students must select from the approved list:

- TWO (2) advanced sorting algorithms **OR**
- TWO (2) advanced searching algorithms

a) Sorting algorithms

- Radix Sort
- Bucket Sort
- Shell Sort
- Heap Sort
- Bingo Sort
- Comb Sort
- Pigeonhole Sort
- Cycle Sort
- Cocktail Sort
- Strand Sort
- Pancake Sort
- Sleep Sort
- Gnome Sort
- Tim Sort

b) Searching algorithms

- Meta Binary Search
- Ternary Search
- Jump Search
- Interpolation Search
- Exponential Search
- Fibonacci Search
- Ubiquitous Binary Search
- Logarithmic Search
- Sublist Search for Linked List
- String searching

Students are NOT allowed to use:

- STL sorting/searching algorithms
- sort()
- binary_search()
- external libraries implementing the algorithms automatically
- All algorithms must be manually implemented.

6. System Development Requirement:

Students must develop a menu-driven C++ application related to a real-world scenario. Comparison between the selected sorting **OR** searching algorithms. Example: If a student decides to implement a sorting algorithm and chooses Gnome Sort and Tim Sort, then find the comparison information, such as the best-case time, average time, worst-case time, and related information. The same case if a student decides to implement a search algorithm.

Choose Titles:

- Student Result Management System
- Smart Library System
- Food Delivery Tracking System
- Online Shopping Catalog
- Hospital Appointment System
- Hotel Reservation System
- Car Rental Management
- Music Playlist Organizer
- Movie Recommendation System
- Warehouse Inventory System
- Student Academic Records
- Online Shopping Products
- Hospital Patient Records
- Hotel Booking Records
- Food Delivery Orders
- Movie Database
- Music Playlist
- Library Book Records
- Warehouse Inventory
- Employee Management

The system must:

- Store two sample dataset: 100 sample records and 500 sample records.
- Allow searching operations or allow sorting operations. Choose ONE only.
- Display execution results clearly,
- Demonstrate the selected algorithms.

7. Algorithm Performance Evaluation:

Students must conduct performance testing using datasets with a minimum 100 and 300 records.

Students must measure:

- Execution time.
- Number of comparisons.
- Number of swaps/movements/iteration.
- Memory usage discussion.
- Algorithm stability discussion.

Students must compare:

- Best case.
- Average case.
- Worst case.
- Implementation difficulty.

Students must present the findings using:

- Tables or
- Charts or
- Graphs.

Students must explain:

- Which algorithm performs better?
- Why performance changes with dataset size?
- Which algorithm is most suitable for their system?

The system must provide meaningful output visualisation, such as:

- Sorted result display or searching path/result display.
- Step-by-step algorithm demonstration.
- Execution statistics.
- Ranking output.
- Categorised records. (100 datasets and 300 datasets).

Students must justify:

- Why the selected algorithms are suitable.
- Situations where the algorithms are NOT suitable,
- How data size affects performance.

8. Integrity Requirements:

Students may use AI tools only for idea exploration, all coding implementations and grammar checking.

However:

- analysis,
- testing,
- comparisons,
- and discussions

must reflect the students' own understanding.

Reminder:
***No late submission is accepted.**
***Copying or other forms of cheating is forbidden (0 marks to all members involved).**
***Do not COMPARE Radix Sort and Heap Sort, as it has been given as a sample in your report template.**